Terraform Lunch and Learn: Speaker Notes

Oct 29, 2026 · @Ryan Tiffany

Before you start

This is a 45-minute session: 25 to 30 minutes of talk, then a 10 to 15 minute live demo. Slides advance on clicks, and **[click]** in the notes marks each advance.

Part	Slides	Minutes
Opening	1-2	about 1
1. The problem	3-5	3-4
2. What Terraform is	6-9	4-5
3. Repeatability	10-14	5
4. State as source of truth	15-24	7-8
5. Auditability	25-28	4-5
6. Road ahead, recap, questions	29-31	3, plus questions
Live demo	32	10-15

If you run long, cut slide 13 (modules) and slide 28 (validation) first.

Opening

Slide 1: Infrastructure as Code on IBM Cloud

Welcome. 45 minutes: 25 to 30 of slides and questions, then a 10 to 15 minute live demo where we stand up a VPC and a server, change it, plan and apply.

Rough pacing for the talk: problem 3-4 min, what Terraform is 4-5, repeatability 5, state 7-8, auditability 4-5, road ahead and recap 3.

This is session 1 of a series. Today is the foundation (Terraform). Each later session answers the pain left by the one before: Schematics, then Deployable Architectures, then Projects.

Slide 2: Three ideas for today

- **[click]** Repeatability: same code, same result.
- **[click]** State: the file that makes Terraform work, and the one new idea for most of you.
- **[click]** Auditability: you can prove what changed.
- **[click]** Then we do it live. No background in Terraform needed.

1. The problem (3-4 min)

Slide 3: "Build me the same POC environment. Again."

Open with the question and wait for hands.

Most people who have built a POC have rebuilt one. That is the problem this whole series is about.

Slide 4: Here is what that looks like in the portal

Say the steps out loud as each one appears. Do not rush it; the length is the point.

- **[click x9]** one per step.
- **[click]** That is more than 20 clicks, and if you create the subnet before the VPC, it fails.

Slide 5: The second build never matches the first

- **[click]** Drift and snowflakes: every environment ends up slightly different.
- **[click]** Nobody remembers what was clicked.
- **[click]** When the customer asks what you deployed, the answer is a screenshot.
- **[click]** Transition: instead of telling IBM Cloud how, just tell it what you want.

2. What Terraform is (4-5 min)

Slide 6: Declarative: describe the end state, not the steps.

This is the key line of the whole talk. Leave it on screen for a moment.

- **[click]** End state underlined: the result you want.
- **[click]** Steps crossed out: you stop being the orchestrator.

Slide 7: Portal versus Terraform

Left side: read each instruction in a bored, step-by-step voice. [click x4]. You track what is done.

Right side: you describe the result. You never say "first, then". Terraform sees that the subnet refers to the VPC, so it builds the VPC first.

Analogy for the next slide: turn-by-turn directions versus a blueprint.

Slide 8: Directions or a blueprint

- **[click]** Verbal instructions: turn-by-turn directions.
- **[click]** Blueprint: you describe the end result, and the contractor checks what already exists.
- **[click]** The bonus point: the second run does nothing. I will show this in the demo when apply says "No changes".

Slide 9: Four words are all you need today

Do not teach syntax. Match each word to a block of code.

- **[click]** provider: IBM Cloud, in this region.
- **[click]** resource: the server we want.
- **[click]** variable: the knobs. Change the value, not the code.
- **[click]** output: what we get back when it is built.

Declarative is what makes the next three ideas possible: repeatability, state and audit.

3. Repeatability (5 min)

Slide 10: Same file. Same result. Every time.

Callback to the 20+ clicks from the start of the talk.

Slide 11: Change the inputs, not the code

Show the idea, not the syntax. Four variables: region, zone, name prefix, server profile.

The point: the code does not change between environments. Only the values do.

Slide 12: One codebase, three environments

- **[click]** Dev: small profile, one zone.
- **[click]** Test: same code, different prefix.
- **[click]** Customer POC: same code, the customer's name as the prefix.
- **[click]** Only the values change.

Demo callback: change name_prefix and show plan proposing a completely new set of resources from the same code.

Slide 13: Modules: write it once, call it anywhere

One sentence only: a module is a reusable chunk. Write "a VPC with subnets" once, and call it from anywhere.

Do not go deeper. If you are running long, cut this slide.

- **[click]** Modules are the bridge to Deployable Architectures.

Slide 14: Why this matters when you sell

Say these three out loud; they are the seller takeaways.

- **[click]** Faster POCs.
- **[click]** Consistent customer environments.
- **[click]** Reusable patterns.
- **[click]** Transition: repeatable only works if Terraform knows what it already built. That is the next idea, and it is the big one.

4. State as source of truth (7-8 min)

Slide 15: "I want a VPC." Does it already exist?

Take this section slowly. It is the concept least familiar to this audience.

- **[click]** Did I make it?
- **[click]** Which one is it?
- **[click]** Terraform cannot look at the cloud and guess which VPC is "mine".

Slide 16: The state file is Terraform's memory

Trimmed example; a real state file holds every attribute of every resource. A JSON file that maps each thing in your code to the real resource ID in IBM Cloud.

- **[click]** The type and name from your code.
- **[click]** The real ID in IBM Cloud. Highlight this one.
- **[click]** The values Terraform last recorded.

Do not cat the whole state file on screen. It can contain sensitive values.

Slide 17: See it for real

This is the same state file, read through two safe commands. It is also what you will run in the demo.

- **[click]** state list: every resource Terraform is tracking, by its name in the code. Four resources, four lines.
- **[click]** state show: one resource, with the real ID and the recorded values. Compare it with the JSON from two slides ago.
- **[click]** The ID is the link between the name in your code and the real server in IBM Cloud.

Why use these instead of opening the file: state show hides values marked sensitive, so it is safe to put on a screen. They are still stored in the state file itself, which is why the file needs protecting.

Slide 18: The loop: write, plan, apply

Put this on one slide: write, plan, apply.

- **[click]** write: describe the end state.
- **[click]** plan: Terraform compares and shows the difference. Nothing changes. This is the safe step.
- **[click]** apply: executes only that difference, then updates state.
- **[click]** Init runs once to download the IBM Cloud provider; then it is the loop.

The interactive State Loop page walks through this with the desired and current state highlighted.

Slide 19: Plan compares three things

Desired is blue; current is purple. Keep the colors consistent with the interactive page.

- **[click]** Desired: your code, what you want.
- **[click]** Recorded: the state file, what Terraform remembers.
- **[click]** Actual: what really exists in IBM Cloud.
- **[click]** The plan is the difference between desired and current. Only the server changes.

Plan is also a drift detector: if someone changes something in the portal, plan reveals it.

Slide 20: Why state is the source of truth

- **[click]** If it is not in state, Terraform does not know it exists and will not manage it.
- **[click]** This is why running apply twice gives "No changes". Tie back to the idea in section 2.
- **[click]** It is how Terraform knows what to delete when you remove a line of code.

Slide 21: Code says X. State says Y. Plan fixes it.

Use the exact example from the demo: bump the VSI profile.

Code says bx2-4x16. State says bx2-2x8. Plan reads the ID from state, finds that server, and proposes changing one attribute. The other resources are left alone.

Slide 22: The honest part: where state goes wrong

- **[click]** Local state: one laptop. If it dies, the state is gone.
- **[click]** Sharing: two people, two states, a collision.
- **[click]** Drift: someone changes something in the portal. State and reality disagree. Terraform plan is also a drift detector.
- **[click]** Secrets: state can contain sensitive values.

Slide 23: Who stores state safely, locks it, and controls who sees it?

Land this clearly. Everything that went wrong on the last slide is what Schematics fixes: it stores the state, locks it so two people cannot apply at once, and controls who can see it.

Next is a short checkpoint on state, before section 5.

Slide 24: Does state make sense?

Stop here for a minute or two. Ask the room, or answer them yourself if it is quiet.

- **[click]** Delete a line of code: Terraform sees a resource in state that is no longer in the code, so the plan proposes to destroy it.
- **[click]** Apply twice: the code, the state and the cloud already agree, so there is nothing to change.
- **[click]** The laptop dies: if the state was only on that laptop, Terraform no longer knows what it built, and would try to create everything again. That is the problem Schematics solves.
- **[click]** Invite questions.

Transition to section 5: state and Git together give you the third idea. You can prove what changed, who changed it and when.

5. Auditability (4-5 min)

Slide 25: What changed, who changed it, and when?

Open with the questions every customer, auditor and security team eventually asks.

- **[click]** What changed? In the portal, a screenshot if someone took one. With Terraform, the diff in the commit.
- **[click]** Who changed it? The author of the commit and the person who approved the pull request.
- **[click]** When? The commit timestamp.
- **[click]** Can we undo it? Revert the commit and apply. Be careful with this one: it undoes the code, so check the plan first.

Three ways Terraform gets you there: code in Git, the plan as a change record, and validation before anything is built.

Slide 26: The code lives in Git

Nothing exotic here. Infrastructure is now just files, so everything you already know about source control applies.

- **[click]** A real history, with messages that say why. The diff is one line: the server profile.

- **[click]** History: every change, with author and time.
- **[click]** Pull requests: a second pair of eyes before it merges. Approvals are recorded.
- **[click]** Rollback: revert the commit, then plan and apply.

If someone asks about the portal: console changes do not show up here. That is drift, and the next plan reveals it.

Slide 27: The plan is the change record

The plan you saw in section 4 is also a review document: it says exactly what will happen before anything does.

- **[click]** propose: the code change goes into a pull request.
- **[click]** review: the plan output is attached. A reviewer can see "1 to change, 0 to destroy" before approving. That is much easier to review than a screenshot of a console.
- **[click]** apply: merge, then apply. You can save a plan to a file with plan -out and apply that exact file, so what you applied is what was reviewed.
- **[click]** Approve what will actually happen.

Slide 28: Catch mistakes before anything is built

Keep this short. The idea: the cheapest place to catch a mistake is before anything exists.

Variable validation is a rule you write next to the variable. If someone passes a profile family you do not allow, Terraform stops with your message, before it plans anything.

- **[click]** The error: it stops at once, with your message.
- **[click]** fmt: formatting, so diffs stay small and readable.
- **[click]** validate: checks the configuration for syntax and reference mistakes without creating anything.
- **[click]** Variable validation: your own rules for what inputs are allowed.

Deeper policy checks come later, with Schematics. Do not go further here.

6. The road ahead (3 min)

Slide 29: Each step answers the pain of the one before

One slide to hold the whole series. Each step is the answer to the pain left by the previous one.

- **[click]** Terraform, today: it answered click-ops.
- **[click]** Schematics, next session: it runs Terraform as a managed service. State is stored centrally, with locking, IAM access control and logs. That answers the one-laptop problem from section 4.
- **[click]** Deployable architectures: predefined solutions such as a VPC landing zone, so teams stop writing and wiring their own from scratch.
- **[click]** Projects: a named collection of configurations. You pick a deployable architecture, add it to a project and set its inputs; the same architecture can have one configuration each for dev, test and production, even across accounts. Changes are validated (commit checks, vulnerability scans, cost estimates), approved by a project admin or editor, then deployed. Projects use Schematics workspaces underneath to hold the state and run Terraform, so people should not edit those workspaces directly. That answers the problem of managing many deployments. The service itself is free; the resources it deploys are billed as usual.

Tease the next session: Schematics.

Slide 30: Three ideas to take away

- **[click]** Repeatability: the same code gives the same environment.
- **[click]** State: the record Terraform trusts, and the reason plan knows what to change.
- **[click]** Auditability: every change is in Git, and every plan can be reviewed.

Next: questions, then the demo.

Slide 31: Questions before we build one?

Leave this up while you take questions. If it is quiet, pick one of these to ask the room:

- Where would a repeatable environment save you the most time with a customer?
- Who on your accounts has been burned by a hand-built environment?

When you are ready, move to the demo.

Live demo (10-15 min)

Slide 32: VPC and server, then change it

Run it live.

1. **Show the code (2-3 min):** provider, VPC, subnet, server, and the variables. Point at the variables.
2. **init and plan (2-3 min):** read the plan out loud. "N to add."
3. **apply (2-3 min):** resources appear in the console. Run terraform state list and point at the IDs. Do not cat the whole state file.
4. **Change one thing (2-3 min):** bump the server profile. The plan reads the ID from state and shows one change. Say whether it is an in-place update or a replacement.
5. **Wrap (2-3 min):** git diff is the audit trail. Optional destroy.

Watch for these callbacks:

- Section 2: a second apply says "No changes".
- Section 3: change name_prefix and the plan proposes a whole new set of resources.
- Section 4: the plan reads from state.
- Section 5: the git diff.

Tips: pre-run init, and keep a pre-built copy as a fallback. Use a small profile and a single zone so apply finishes fast. Keep secrets in fnox or environment variables, never on screen.